CivicTrack: Project Report at the One-Month Milestone

Project: CivicTrack, a municipal issue-reporting and accountability platform **Duration:** 8 weeks in total, with a milestone review at the end of week 4 **Status at this review:** all week 1–4 deliverables are built, tested and deployed

Live website	https://civic-track-six.vercel.app (https://civic-track-six.vercel.app)
Live API	https://civictrack-api.onrender.com (https://civictrack-api.onrender.com)
API documentation (Swagger)	https://civictrack-api.onrender.com/swagger-ui/index.html (https://civictrack-api.onrender.com/swagger-ui/index.html)

> *The API runs on a free hosting tier that sleeps when idle. The first request after a quiet period can take up to about a minute. After that, responses are immediate.*

Contents

1. The problem and the idea
2. Milestone checklist
3. Technology stack and why each piece was chosen
4. System architecture
5. Database design
6. Core feature 1: the geo-clustering engine
7. Core feature 2: the issue lifecycle (state machine)
8. Core feature 3: priority scoring
9. Core feature 4: SLA deadlines and escalation
10. Authentication and roles
11. REST API
12. Frontend
13. Photo pipeline
14. Seed data and ground-truth labels
15. Testing
16. Deployment and CI/CD
17. Key design decisions
18. Problems we hit and what we learned
19. What is not built yet (month 2)
20. Numbers at a glance
21. Likely questions and answers

1. The problem and the idea

The problem

Municipal complaint systems usually fail in three ways:

1. **Duplicates drown the signal.** Twenty people report the same pothole and the department receives twenty separate low-priority tickets. Nothing marks the pothole as the one many people care about.
2. **Nothing forces a deadline.** A complaint can sit untouched indefinitely, and no one above the responsible officer ever finds out.
3. **Departments mark their own work as done.** A ticket gets closed as "resolved" whether or not the problem was actually fixed.

What CivicTrack does about each

Problem	CivicTrack's answer
Duplicates	A **geo-clustering engine** merges reports of the same problem, in the same category and nearby, into **one issue** in real time. Twenty reports become one urgent ticket that shows "20 people reported this".
No deadlines	Every issue gets an **SLA deadline** from its category and priority. When the deadline passes, the issue **escalates automatically** up a four-level ladder.
Self-certified fixes	**No staff member can mark an issue resolved.** Staff can only submit it for citizen verification, and the system resolves it after citizens confirm.
No transparency	A **public dashboard** and public issue pages show overdue counts and progress without requiring a login.

A citizen can report a problem, with photo and GPS location, in under twenty seconds and **without creating an account**.

2. Milestone checklist

The project plan (`docs/civictrack-project-report.md`, §17) defines the first four weeks as follows. Each item was checked against the code and the running system.

Week 1: setup and schema □

Deliverable	Where it is
Monorepo (`backend/`, `frontend/`, `docs/`)	Repository root
CI pipeline	`.github/workflows/ci.yml`
Flyway database migrations	`backend/src/main/resources/db/migration/V1–V4`
JPA entities	`backend/src/main/java/com/civictrack/**`
`GeoFactory`, the one place lat/lng become a geometry point	`common/geo/GeoFactory.java`, tested by `GeoFactoryTest`

Deliverable	Where it is
PostGIS locally (Docker) and in production (Supabase)	`docker-compose.yml`, deployed database
Deployed health check	`/actuator/health` returns `{"status":"UP"}`
Ward-containment canary test	`WardContainmentIT`

Week 2: clustering engine and seed generator □

Deliverable	Where it is
Spatial candidate query	`ClusteringService`, `ClusteringQueryPlanIT` (checks that the spatial index is used)
Accuracy-weighted centroid	`CentroidMath`, `CentroidMathTest`
Adaptive merge radius	`ClusteringService`
Cluster extent cap	`ClusteringService`, `ClusteringServiceIT`
Re-reading distances after locks are granted	`ClusteringService` (DD-003)
Advisory locking for concurrent reports	`ClusteringService`, `ClusteringConcurrencyIT` (20 simultaneous submissions produce 1 issue)
Deterministic seed corpus with ground-truth labels	`seed/SeedDataGenerator.java`, `backend/ground_truth_labels.csv`

Week 3: state machine, SLA and authentication □

Deliverable	Where it is
Transition policy and guards	`issue/policy/TransitionPolicy`, `IssueAccessGuard`, `TransitionPolicyTest`
Status history (audit trail)	`issue_status_history` table, `IssueStatusService`
Escalation ladder	`sla/escalation/`, `EscalationLadderIT`
Sweep job with ShedLock	`SlaEscalationJob`, `EscalationJobLockIT`
Priority ageing	`PriorityRecomputer`, `PriorityAgeingIT`
Idempotency tests	`EscalationIdempotencyIT`
JWT authentication and role-based access control	`user/auth/`, `AuthApiIT`

Week 4: frontend core and deployment □

Deliverable	Where it is (live)
Report flow	`/report` → `/report/success/[ref]`
Map	`/map`
Issue detail	`/issues/[id]`
Staff queue	`/staff/queue`, `/staff/issues/[id]`
Read-only cluster inspector	`/issues/[id]/cluster`
Public URLs live	Vercel (frontend) and Render (API)
Seeded data	110 issues from 250 reports on the live database

Verification run for this review: backend 172 tests passed with 0 failures, frontend 58 tests passed, TypeScript typecheck clean, and every live page and API endpoint listed above returned HTTP 200.

---

3. Technology stack and why each piece was chosen

Backend

Technology	Version	Why
Java	21 (LTS)	Records for DTOs, switch expressions and pattern matching; long-term support
Spring Boot	3.5	Industry-standard web, data and security framework
Spring Data JPA + Hibernate Spatial	—	Maps entities to tables, including PostGIS geometry columns
PostgreSQL + PostGIS	PG 17 (production), PostGIS 3.x	**The whole project depends on spatial queries.** PostGIS provides `ST_DWithin`, `ST_Distance` on geography and spatial indexes.
Flyway	—	Versioned, repeatable schema migrations; the database is never changed by hand
Spring Security OAuth2 Resource Server	—	Validates JWTs using Spring's own `JwtDecoder`. We deliberately **did not hand-write an auth filter**, because hand-rolled authentication code is where security bugs live (DD-007).
ShedLock	—	Makes sure the scheduled SLA sweep runs on only one server instance at a time
springdoc-openapi	—	Generates Swagger UI API documentation automatically
Lombok	—	Removes boilerplate getters and setters on entities only
Spring `ProblemDetail` (RFC 7807)	built in	Standard JSON error responses without an extra library

Frontend

Technology	Version	Why
Next.js (App Router)	16	Server-side rendering, so the landing page's live overdue count is in the initial HTML; route handlers keep the refresh token away from JavaScript
React	19	UI library
TypeScript	5	Type safety across the API boundary
Tailwind CSS	4	Design tokens, light and dark themes

Technology	Version	Why
TanStack React Query	5	Server-state caching; a status change refreshes every view that shows the issue
React Hook Form + Zod	—	Form handling, with validation rules in one schema that mirrors the server
Leaflet	1.9	Open-source maps with no API key or billing, loaded only when a map is actually needed
Vitest	5	Unit tests

Infrastructure

Service	Used for
Supabase	Managed PostgreSQL with PostGIS (free tier)
Render	Runs the backend Docker container (free tier)
Vercel	Hosts the Next.js frontend
Cloudinary	Photo storage and image transformation
GitHub Actions	CI: tests on every push, deploy on `main`, and a keep-alive ping
Docker / Testcontainers	Local database and real-PostGIS integration tests

Things we deliberately did not use, and why

Not used	Reason
H2 in-memory database for tests	H2 has no PostGIS, so spatial SQL cannot be tested on it. Every integration test runs against a real PostGIS container.
`jjwt` or a custom JWT filter	Spring Security already implements JWT validation correctly
MapStruct	With only about fifteen DTOs, Java records with a static `from(entity)` method are simpler
Google Maps	Billing and an API key; Leaflet with OpenStreetMap is free

---

4. System architecture

graph TB subgraph browser["Citizen / staff browser"] UI["Next.js 16 + React 19"] end subgraph vercel["Vercel"] SSR["Server components
(landing page, SSR)"] AUTHH["/api/auth/* route handlers
hold refresh token in httpOnly cookie"] end subgraph render["Render (Docker)"] API["Spring Boot 3.5 / Java 21"] CL["ClusteringService"] ST["IssueStatusService
(only writer of status)"] SLA["SLA sweep job
every 5 min, ShedLock"] end subgraph supabase["Supabase"] DB[("PostgreSQL 17 + PostGIS")] end CDN["Cloudinary
(photos)"] UI -->|"REST + bearer token"| API UI -->|"sign in / refresh"| AUTHH --> API SSR --> API UI -->|"photo upload"| CDN API --> CL --> DB API --> ST --> DB SLA --> DB

Backend package structure (by feature, not by layer)

```
com.civictrack
├── clustering     report-merging engine (the core algorithm)
├── report         report ingestion API
├── issue          Issue entity, lifecycle, transition policy, priority
├── sla            deadlines, sweep job, escalation ladder
├── verification   citizen verification (schema and quorum service; screens in month 2)
├── user           accounts, login, JWT, roles
├── department     organisation chart
├── ward           ward boundaries (polygons)
├── category       categories and their tunable parameters
├── dashboard      public summary figures
├── publicapi      read-only public endpoints
├── me             "my reports" for signed-in citizens
├── media          Cloudinary orphan cleanup
├── seed           demo data generator
├── config         security, scheduling, clock
└── common         GeoFactory, error handling
```

We packaged by feature so that everything about clustering is in one folder, not spread across `controllers/`, `services/` and `repositories/`.

Why authentication is split this way

- The **access token** lives only in React memory and disappears when the tab closes.
- The **refresh token** never reaches JavaScript. Vercel route handlers store it in an **httpOnly cookie** and exchange it server-side.
- Reason: a long-lived token in `localStorage` could be stolen by any injected script, which would amount to a 30-day account takeover.

---

5. Database design

Eleven tables are created by migration `V1__baseline.sql`:

Table	Purpose
`departments`	Organisation chart, with parent departments (for example, Roads sits under Public Works Zone)
`wards`	City wards as **polygons** (`MultiPolygon`, SRID 4326)
`categories`	Problem types and **every tunable number** (merge radius, SLA hours, severity, and so on)
`users`	Accounts and roles
`issues`	One row per real-world problem, the result of clustering
`reports`	One row per citizen submission, each linked to an issue

Table	Purpose
`issue_status_history`	Audit trail of every status change: who, when, from, to, and why
`escalation_events`	Every escalation, unique per issue and level
`verifications`	Citizen "fixed / not fixed" votes (used in month 2)
`notifications`	Notification records (used in month 2)
`shedlock`	Lock table for the scheduled job

erDiagram DEPARTMENTS ||--o{ CATEGORIES : owns DEPARTMENTS ||--o{ DEPARTMENTS : "parent of" WARDS ||--o{ ISSUES : contains CATEGORIES ||--o{ ISSUES : classifies ISSUES ||--o{ REPORTS : "clusters" ISSUES ||--o{ ISSUE_STATUS_HISTORY : "audit trail" ISSUES ||--o{ ESCALATION_EVENTS : escalates ISSUES ||--o{ VERIFICATIONS : "citizens judge" USERS ||--o{ REPORTS : "reported by (optional)"

Reference data (migration `V2`)

Departments: Public Works Zone (Roads and Infrastructure; Water Supply and Drainage; Street Lighting and Electrical) and Civic Services Zone (Sanitation and Waste; Animal Control).

Wards: four wards in Ludhiana (Civil Lines North, Bharat Nagar, Model Town South, Sarabha Nagar), stored as polygons.

Categories and their tunable parameters:

Category	Merge radius (m)	SLA (hours)	Severity weight	Extent cap multiplier	Ambiguous-band action
Pothole	25	72	15	2.0	merge and flag
Road Damage	60	120	12	2.5	merge and flag
Damaged or Missing Signage	20	168	6	2.0	merge and flag
Streetlight Out	30	96	10	2.0	merge and flag
Garbage Accumulation	50	48	12	2.0	merge and flag
Illegal Dumping	50	48	14	2.0	merge and flag
Water Leak	40	24	20	2.0	merge and flag
Blocked Drain	40	24	18	2.0	merge and flag
Open or Missing Manhole Cover	20	**6**	**40**	1.5	**split and flag**
Stray Animal	120	24	16	2.0	**split and flag**

Why these numbers are in the database rather than in Java code: changing a merge radius is a single SQL `UPDATE`, not a code change and redeploy. The answer to "how did you choose 25 m?" is configuration with a documented reason.

Design choices in the schema

- **`sum_w`, `sum_wx`, `sum_wy` columns on `issues`** store running totals, so adding a report to a cluster updates the centroid in **O(1)** instead of re-reading every report in it.
- **`max_member_dist_m`** makes the extent-cap check a single comparison.

- **`paused_seconds`** records time the SLA clock was stopped, separately from `due_at`, so the original deadline stays visible in the audit trail.
- **`public_ref`** (for example `CT-2026-000432`) is a readable ticket number citizens can quote.
- A **spatial GiST index** makes nearby-issue lookups fast. `ClusteringQueryPlanIT` checks the query plan to confirm the index is actually used (DD-010).

---

6. Core feature 1: the geo-clustering engine

This is the project's main technical contribution.

Goal

When a report arrives, decide in real time: **is this a new problem, or another report of an existing one?**

Step by step

sequenceDiagram participant C as Citizen participant API as ReportController participant CL as ClusteringService participant PG as PostGIS C->>API: POST /api/v1/reports {photoUrl, lat, lng, accuracy, category} API->>CL: ingest(command) CL->>PG: pg_advisory_xact_lock(hash(category, ~200 m grid cell)) CL->>PG: find nearby open issues of same category (ST_DWithin), lock rows CL->>PG: re-read locked rows and recompute distances CL->>CL: compute effective radius, choose decision band CL->>CL: check extent cap CL->>PG: update centroid (O(1)) or create new issue CL-->>API: decision API-->>C: 201 "You are the 4th person to report this"

1. Candidate search

Find open issues in the **same category** within range, using `ST_DWithin(...::geography)`. All distances are measured on the Earth's surface (geography), **never in flat latitude/longitude degrees**, which would be wrong.

2. Accuracy-weighted centroid

Every report has a GPS accuracy (for example ±10 m). Each report gets the weight

$$w = \frac{1}{\text{accuracy}^2}$$

so a precise GPS fix pulls the issue's location more than a vague one. The centroid is the weighted average of all report positions, kept up to date with running sums (`sum_w`, `sum_wx`, `sum_wy`).

3. Adaptive merge radius

$$R_{\text{eff}} = R_{\text{cat}} + \tfrac{1}{2}\,\text{accuracy} + \tfrac{1}{2}\,\sigma, \qquad \sigma = \frac{1}{\sqrt{\sum w}}$$

- R_{cat} is the category's base radius (25 m for a pothole).

- The incoming report's own GPS inaccuracy widens the radius.
- \(\sigma\) is the cluster's own positional uncertainty. **It shrinks as more reports arrive.** Twenty reports at 10 m accuracy give σ ≈ 2 m, while two reports give σ ≈ 7 m.
- The effect: a well-evidenced cluster gets a **tighter** radius, with nothing to tune by hand.

4. Three decision bands

Distance from issue centroid	Decision
≤ R_eff	**Merge** (high confidence)
Between R_eff and 1.5 × R_eff	**Ambiguous.** Depends on the category (see below)
> 1.5 × R_eff	**New issue**

For the ambiguous band, each category decides (DD-002):

- **Merge and flag** (potholes and similar): a wrong merge costs one click to split later.
- **Split and flag** (open manholes, stray animals): merging would **hide a second dangerous hazard** behind a counter. Stray animals move, so being nearby is weaker evidence that two reports are the same animal.

Either way, the decision is flagged for supervisor review.

5. Extent cap (preventing "cluster drift")

The problem: along a long road, each new report could be just within range of the moving centroid, so a cluster could creep kilometres along the road through a chain of individually valid merges.

The fix (DD-001): a cluster may never span more than `max_extent_multiplier × R_cat`. If a merge would exceed that, the report becomes a new issue (`SPLIT_EXTENT_CAPPED`).

6. Concurrency: three phones, one pothole

If three people report the same pothole at the same moment, a naive system creates three issues. We prevent this with:

1. **A transaction-scoped PostgreSQL advisory lock** on a hash of (category, ~200 m grid cell). Simultaneous reports of the same place wait in line, while reports elsewhere in the city proceed in parallel.
2. **Two-phase candidate lookup.** PostgreSQL sorts rows *before* granting row locks, so distances read in the first query may be stale. We lock the rows, then **re-read** them, and only the re-read values decide the merge (DD-003).
3. **Short critical section.** Display names (ward, department) are read **after** the transaction commits, so locks are held as briefly as possible (DD-024).

Proof: `ClusteringConcurrencyIT` fires **20 concurrent submissions** of the same defect and asserts that **exactly one issue** results.

7. Every decision is auditable

Each report row stores `cluster_decision` (`NEW_ISSUE`, `MERGED`, `MERGED_LOW_CONF`, `SPLIT_LOW_CONF`, `SPLIT_EXTENT_CAPPED`), the distance and the effective radius used. **The cluster inspector screen** (`/issues/[id]/cluster`) shows this visually: every report's position, the centroid, the radius, and why each report merged.

---

7. Core feature 2: the issue lifecycle (state machine)

Nine statuses: `NEW`, `ACKNOWLEDGED`, `ASSIGNED`, `IN_PROGRESS`, `PENDING_VERIFICATION`, `RESOLVED`, `REOPENED`, `CLOSED`, `REJECTED`.

stateDiagram-v2 [*] --> NEW : citizen reports NEW --> ACKNOWLEDGED : staff ACKNOWLEDGED --> ASSIGNED : supervisor only ASSIGNED --> IN_PROGRESS : assignee REOPENED --> IN_PROGRESS : staff IN_PROGRESS --> PENDING_VERIFICATION : proof photo + note (≥20 chars) PENDING_VERIFICATION --> RESOLVED : SYSTEM, citizens confirm PENDING_VERIFICATION --> REOPENED : SYSTEM, citizens say not fixed RESOLVED --> REOPENED : SYSTEM, problem recurs RESOLVED --> CLOSED : SYSTEM or ADMIN, after 7 days NEW --> REJECTED : supervisor, reason ≥20 chars ACKNOWLEDGED --> REJECTED : supervisor ASSIGNED --> REJECTED : supervisor IN_PROGRESS --> REJECTED : supervisor REOPENED --> REJECTED : supervisor CLOSED --> [*] REJECTED --> [*]

The key rule: staff cannot resolve their own work

There is **no transition** from any staff-reachable state to `RESOLVED` or `CLOSED`. Staff go as far as `PENDING_VERIFICATION` and stop. This is enforced in three independent places:

1. **`TransitionPolicy` refuses to start** if its transition table ever contains such a transition, so a bad edit crashes the application at startup instead of shipping.
2. **`TransitionPolicyTest`** tries a staff move to `RESOLVED` from **all nine states** and requires all nine to be refused.
3. **The staff UI** never shows such a button.

A single writer for status

Only `IssueStatusService` can change an issue's status. `Issue.setStatus` is package-private, so **the Java compiler itself** prevents other code from changing status and skipping the audit-trail entry. Every change writes a row to `issue_status_history`.

Two authorisation layers (DD-020)

1. **Role check:** is this role allowed this transition at all?
2. **Ownership check** (`IssueAccessGuard`): for example, only the *assigned* staff member can start work on an issue.

8. Core feature 3: priority scoring

```
score = severity_weight(category)
      + 12 × log2(1 + distinct_reporters)
      + 0.15 × age_hours          (excluding paused time)
      + 20 × escalation_level
      + 15 × reopen_count
```

Score	Priority
≥ 80	CRITICAL
≥ 50	HIGH
≥ 25	MEDIUM
< 25	LOW

Two choices that prevent gaming

- **Distinct reporters, not report count.** One person submitting twenty times counts once.
- **Logarithmic, not linear.** Going from 1 to 5 reporters matters a lot; going from 40 to 45 barely matters. Without the log, one organised street could dominate the whole city's queue by out-reporting everyone else.

Priority ageing

`PriorityRecomputer` re-scores open issues on every sweep, so an old ignored issue **rises in priority over time** (DD-004). Time spent waiting for citizen verification is **excluded**, so an issue does not climb the queue while the department is waiting on citizens (DD-021).

---

9. Core feature 4: SLA deadlines and escalation

Deadline calculation

```
deadline = category SLA hours × priority multiplier
```

Priority	Multiplier	Example: pothole (72 h)
CRITICAL	0.25	18 h
HIGH	0.5	36 h
MEDIUM	1.0	72 h
LOW	1.5	108 h

If priority rises, the deadline can move **earlier**, never later.

The clock pauses

While an issue is `PENDING_VERIFICATION`, the SLA clock **stops**, because the department is waiting on citizens and should not be penalised for that. The UI shows "Clock paused" instead of a countdown.

Escalation ladder

A background job (`SlaEscalationJob`) runs **every 5 minutes**. An issue is breached when `now > due_at + paused_seconds` and the clock is running.

graph LR B["Deadline breached"] --> L1["Level 1
Department head"] L1 --> L2["Level 2
Parent department head"] L2 --> L3["Level 3
Ward officer"] L3 --> L4["Level 4
Administrator
(terminal)"]

- Levels 1–2 walk **up the organisation chart**. Level 3 hands the issue to the **ward officer**, who is responsible for a place rather than a department (DD-005).
- Level 4 is the top. Issues there stay visible on a chronic-breach list rather than being hidden.

Safe under duplicate or concurrent runs (idempotency)

The sweep may run twice, or on two servers at once. Three layers make sure an issue never escalates twice for one breach:

1. **ShedLock:** only one instance runs the job at a time.
2. **`FOR UPDATE SKIP LOCKED`:** parallel workers pick up different issues.
3. **A unique constraint** on the escalation event, plus a **compare-and-swap** update on `escalation_level`.

`EscalationIdempotencyIT` and `EscalationJobLockIT` test this.

Testable time

All time-dependent logic reads time from an **injected `java.time.Clock`** rather than calling `Instant.now()`. Tests move the clock forward, for example by 3 days, and check the escalation instantly, with no slow or flaky `sleep()` calls.

---

10. Authentication and roles

Role	Can do
Anonymous	Report a problem, view public issues, map and dashboard
`CITIZEN`	Everything above, plus "My reports" (and verification votes in month 2)
`STAFF`	Staff queue, acknowledge, start work, submit for verification
`SUPERVISOR`	Assign, reject (with reason)
`ADMIN`	Close, and top of the escalation ladder
`SYSTEM`	Automatic transitions (resolve, reopen, auto-close)

- **Anonymous reporting is allowed on purpose (DD-017).** Requiring registration would collect fewer reports, and disproportionately from people who already trust the municipality. If a signed-in user reports, their identity is attached from the verified token and **never taken from the request body**, so nobody can file a report in someone else's name.
- Login returns a **JWT access token** (validated by Spring Security's resource server) and a **refresh token**, which the frontend keeps in an httpOnly cookie.
- Errors use the standard **RFC 7807 `ProblemDetail`** JSON format.

---

11. REST API

All endpoints sit under `/api/v1`. Full interactive documentation is at **`/swagger-ui/index.html`**.

Area	Endpoint	Access
Auth	`POST /auth/register`, `POST /auth/login`, `POST /auth/refresh`	Public
Reports	`POST /reports`	Public (identity attached if signed in)
Public issues	`GET /public/issues`	Public
	`GET /public/issues/bbox` (issues within the visible map area)	Public
	`GET /public/issues/{id}`	Public
	`GET /public/issues/by-ref/{publicRef}`	Public
	`GET /public/issues/{id}/history`	Public
	`GET /public/issues/{id}/reports` (used by the cluster inspector)	Public
Dashboard	`GET /dashboard/summary`	Public
Reference data	`/categories`, `/wards`	Public
My reports	`GET /me/reports`	Citizen
Staff queue	`GET /staff/queue?tab=ALL|MINE|UNASSIGNED`	Staff
Lifecycle	`POST /issues/{id}/acknowledge`	Staff
	`POST /issues/{id}/assign`	Supervisor
	`POST /issues/{id}/start`	Assigned staff
	`POST /issues/{id}/submit-for-verification`	Staff
	`POST /issues/{id}/reject`	Supervisor
	`POST /issues/{id}/close`	Admin
Health	`GET /actuator/health`	Public

Illegal status transitions return **HTTP 409 Conflict**; missing permissions return **403**.

---

12. Frontend

Screens

Route	Screen	What it does
`/`	Landing page	Server-rendered, with the live overdue count in the initial HTML
`/report`	Report composer	Pick category → photo (compressed in the browser) → GPS location, or pin placement if GPS is poor → submit
`/report/success/[ref]`	Confirmation	Shows the ticket reference and whether the report merged ("You are the 4th person…")
`/map`	City map	Issues as markers **sized by number of reporters**, with a detail panel; some layers depend on role
`/issues`	Issue list	Public browsable list
`/issues/[id]`	Issue detail	Status, deadline countdown, photo, history timeline
`/issues/[id]/cluster`	**Cluster inspector**	Visual explanation of how reports were merged: positions, centroid, radius, decision per report
`/dashboard`	Public dashboard	Overdue count, total resolved, worst ward and worst department
`/me/reports`	My reports	A signed-in citizen's own submissions
`/staff/queue`	Staff queue	Tabs: All / Mine / Unassigned
`/staff/issues/[id]`	Staff work view	Shows only the actions the server will accept for the current status
Sign in / Register	Dialog	`/login` and `/register` open as a modal dialog

Accessibility: status is shown three ways

Every status is shown with **colour, shape and a word**, never colour alone, so it remains readable for colour-blind users and in greyscale. Checking a greyscale screenshot caught a real bug: `RESOLVED` was missing its left border and was quietly relying on colour.

Performance

- The report composer is designed for **cheap phones on slow connections**. Measured open-to-ticket time is **3.0 s** under simulated 1.6 Mbps, 150 ms latency and 4× CPU throttling.
- **Leaflet (the map library) is not loaded on the report page** unless the user needs to place a pin by hand, so the normal path loads 0 KB of map code.
- Light and dark themes are both supported.

---

13. Photo pipeline

1. The browser **compresses** the photo (a 1200×900 test photo became about 26 KB).
2. The browser **uploads directly to Cloudinary** using an unsigned upload preset, restricted by size and format.
3. The resulting URL is stored with a delivery transformation (`f_auto,q_auto,w_1600,c_limit`), so each browser receives AVIF or WebP at a suitable quality.

4. The URL is sent with `POST /reports`.

Why upload before the database transaction: a slow third-party upload must never happen while the clustering engine holds its lock. **The cost:** an abandoned submission leaves an unused photo, so a **nightly orphan sweep** deletes those (`CloudinaryOrphanSweep`, tested by `CloudinaryOrphanSweepIT`).

Trade-off we accepted (DD-053): unsigned uploads mean the preset name is public. Signed uploads would be the production-grade answer. For a free-tier university project we accepted this, restricted the preset (≈2 MB limit, jpg/png/webp only, a dedicated folder) and documented the decision.

> ***Note for the demo:*** *the seeded demo reports all point at Cloudinary's public sample image, on purpose (DD-037). Real photos uploaded through* `/report` *are stored and displayed properly; this was checked end to end.*

---

14. Seed data and ground-truth labels

- `SeedDataGenerator` (`seed` profile) generates a realistic corpus of **250 reports**, which cluster into **110 issues**, from random seed 42.
- It writes **`ground_truth_labels.csv`**, recording for every report which real-world defect it actually belongs to.
- **Why this matters:** in week 7 we will measure the clustering engine's **precision, recall and F1** by comparing what it merged against these labels. Generating the labels together with the data removes days of manual labelling.
- The deployed data covers **eight of the nine statuses**, so every screen state can be demonstrated. `REOPENED` is intentionally absent because it needs citizen verification, which arrives in month 2.
- **Honest note (DD-046):** the corpus is not fully reproducible run to run, so the exact deployed corpus and its labels are **backed up** (`backups/`).

---

15. Testing

Numbers (run on 13 September 2026)

Suite	Tests	Result
Backend (JUnit 5 + Testcontainers against real PostGIS)	**172** in 27 test classes	all passed
Frontend (Vitest)	**58** in 5 files	all passed
TypeScript typecheck	—	clean

Backend test classes and what each proves

Test	Proves

Test	Proves
`GeoFactoryTest`	Latitude and longitude are never swapped
`WardContainmentIT`	A point inside a ward polygon is matched to that ward (canary for coordinate order and SRID)
`CentroidMathTest`	Weighted centroid and uncertainty maths are correct
`ClusteringServiceIT`	Merge, split, ambiguous-band and extent-cap decisions
`ClusteringConcurrencyIT`	20 simultaneous reports → 1 issue
`ClusteringQueryPlanIT`	The spatial index is used, not a full table scan
`ReportIngestApiIT`, `IngestMetricsIT`	The report API end to end, and its metrics
`TransitionPolicyTest`	No staff path to RESOLVED or CLOSED from any of the 9 states
`IssueLifecycleApiIT`	Every lifecycle endpoint, including illegal moves returning 409
`SlaClockIT`	Deadlines, and the clock pausing and resuming
`PriorityAgeingIT`	Priority rises with age, excluding paused time
`EscalationLadderIT`	Correct person at each escalation level
`EscalationIdempotencyIT`, `EscalationJobLockIT`	No double escalation, even with concurrent sweeps
`AuthApiIT`	Register, login, refresh, role enforcement
`StaffQueueTabIT`, `MyReportsIT`, `PublicApiIT`	Staff, citizen and public read APIs
`DashboardSummaryIT`, `DashboardBreachAgreementIT`	Dashboard figures, and that the three definitions of "breached" agree (DD-028)
`SchemaMigrationIT`, `SeedGuardIT`, `DeployProfileIT`, `ApplicationBootIT`	Migrations apply, seeding can't run where it shouldn't, deploy configuration boots
`CloudinaryOrphanSweepIT`	The orphan-photo cleanup deletes the right assets

How we know the tests are meaningful

Every test that claims to guard a behaviour was checked by breaking that behaviour on purpose and confirming the test failed, then restoring the code. The backup restore self-test is a recorded example: it was run against a dump with the issues missing and correctly went red, even though `pg_restore` itself reported success (DD-045).

We adopted this after finding tests that passed while checking nothing. One asserted the extent cap held in a scenario that never reached the cap. Where breaking something left the suite green, we treated it as a gap and wrote a test that actually covered it.

Harnesses against the running server (`tools/`)

Unit tests cannot catch a frontend that disagrees with the real backend, so we wrote four Playwright/Node scripts that run against the live servers:

Script	Catches
`contract.mjs`	TypeScript types that don't match the JSON the API really returns

Script	Catches
`table.mjs`	A transition the UI offers that the server refuses; walks an issue through the whole lifecycle
`uiwalk.mjs`	The wrong action button shown for a status
`sweep.mjs`	Runtime errors, `NaN` or `undefined` on any page, signed in and signed out

Each of these exists because it caught a real bug that the passing unit tests had missed.

---

16. Deployment and CI/CD

graph LR DEV["git push"] --> GH["GitHub Actions
backend: 172 tests on PostGIS
frontend: lint, types, tests, build"] GH -->|"main branch"| HOOK["Render deploy hook"] DEV -->|"Git integration"| VER["Vercel build"] HOOK --> RENDER["Render
Docker container"] VER --> VERCEL["Vercel
Next.js"] RENDER -->|"session pooler :5432"| SUPA[("Supabase
PostgreSQL + PostGIS")]

CI pipeline (`.github/workflows/ci.yml`)

- **Backend job:** JDK 21, cached PostGIS image, `mvn clean test` (always `clean`, see DD-027). On failure, the error details are published where anyone can read them.
- **Frontend job:** `npm ci` → lint → typecheck → unit tests → production build.
- **Deploy job:** only on `main`, only if both jobs pass, and it triggers the Render deploy hook. Vercel deploys the frontend itself.

Fast cold start: Class Data Sharing (CDS)

- **Problem:** Render's free tier stops the container after 15 minutes idle, and a normal Spring Boot start takes **20–40 seconds**, long enough to lose an audience in a demo.
- **Fix:** the three-stage `Dockerfile` does a **training run at build time** that saves pre-parsed class metadata to a CDS archive, and the runtime loads that archive.
- **Result:** application startup of **3.5 seconds**.
- The build **verifies** the archive with `-Xshare:on`, so a broken archive fails the build instead of silently slowing startup.

Other operational pieces

- **Keep-alive workflow** (`keep-alive.yml`) pings the API every 10 minutes and checks the database answers, reducing cold starts.
- **Connection pool** capped at **8** (Hikari), to fit free-tier database limits, with leak detection turned on.
- **Supabase session pooler (port 5432), not transaction mode.** Advisory locks and prepared statements need a stable session. Getting this wrong cost a day (DD-040).
- **Backups:** `scripts/backup.sh` and `scripts/restore.sh` with `pg_dump`. Dumps are in `backups/`, and restores were actually tested (DD-045).

---

17. Key design decisions

We record every non-obvious decision in `docs/DESIGN-DECISIONS.md`: the problem, why it matters, the chosen fix, and the rejected alternative. There are **57 entries** so far. The most important ones:

#	Decision	In one line
DD-001	Extent cap	Stops clusters creeping along long roads through chained merges
DD-002	Per-category ambiguous-band action	Open manholes split rather than merge, so a second hazard isn't hidden
DD-003	Re-read after locking	PostgreSQL sorts before locking, so the first distances may be stale
DD-004 / DD-021	Priority ageing, excluding paused time	Old issues rise; waiting for citizens doesn't count
DD-005	Escalation leaves the department tree at level 3	The ward officer is accountable across departments
DD-007	Spring Security resource server	No hand-written JWT filter
DD-008	Records instead of MapStruct	Too few DTOs to justify a code generator
DD-010	Spatial GiST index on the geography cast	Keeps the candidate query fast; checked by a test
DD-011	One `GeoFactory`, no distance maths in Java	Prevents lat/lng swaps and flat-earth distance bugs
DD-012	Class Data Sharing	Cold start from 20–40 s down to 3.5 s
DD-017	Anonymous reporting allowed	Lower friction, reaches people who distrust the system
DD-020	Two authorisation layers	Role check plus ownership check
DD-024	Names read after commit	Keeps the locked section short
DD-028	Three definitions of "breached" pinned together	Dashboard, sweep and UI can't disagree
DD-035	Client and server transition tables compared	A unit test copied the same wrong assumption as the code
DD-039	Applied migrations are never edited	Even changing a comment broke startup on existing databases
DD-040	Session pooler	Advisory locks need a stable connection
DD-053	Unsigned client-side photo upload	Accepted trade-off, documented and restricted

Standing rules we followed throughout the project:

1. Every tunable number lives in the `categories` table, not in code.
2. Every distance uses `ST_Distance(...::geography)`, never flat maths.
3. Only `GeoFactory` turns lat/lng into a point.
4. Tests are written together with features, not afterwards.
5. Time comes from an injected `Clock`, so time-based logic is testable.

---

18. Problems we hit and what we learned

Three of the biggest problems had the same shape: **a passing test that was not actually checking anything.**

1. **A stale build directory gave two wrong diagnoses (DD-027).** An old compiled `target/` folder served outdated SQL, and we twice "found" a bug in a query that was actually correct. *Lesson:* CI always runs `mvn clean`.
2. **Client and server disagreed on allowed transitions (DD-035).** Every "Start work" click returned 409. The frontend unit test passed because it was written from the same wrong assumption as the code. *Lesson:* where the frontend mirrors server rules, test against the **real server**. This is why `tools/table.mjs` exists.
3. **Editing an applied migration broke production startup (DD-039).** We changed only a comment in an already-applied migration. Flyway's checksum changed and every existing database refused to start, while the tests stayed green because they always use a fresh database. *Lesson:* applied migrations are never edited, only new ones added.

Other notable issues:

- **Database connection (DD-040):** Supabase's direct host is IPv6-only and Render has no IPv6, so we had to use the session pooler. The wrong pooler mode silently breaks locks.
- **CI red for a week (DD-048, DD-051, DD-052):** tests passed locally but failed in CI because of test ordering and an unnecessary Testcontainers helper container. We made CI publish readable failure details so the cause could be read rather than guessed.
- **Backups that weren't backups (DD-045):** `pg_restore` reported success on a dump that did not restore properly. We now test restores.
- **Seed data showed only 2 of 9 statuses (DD-047):** 800 rows of scrolling had hidden this. We reduced the corpus to 250 reports covering 8 statuses.

---

19. What is not built yet (month 2)

Everything below belongs to weeks 5–8 of the original plan; none of it was a week 1–4 deliverable.

Not built yet	Planned week
Citizen verification screen and vote endpoint (the database schema and quorum service already exist)	Week 5
Verification timeout sweep, reopen on rejection, flagging closures without verification	Week 5
Notification centre (the table exists, nothing writes to it yet)	Week 5
Supervisor tools: assignment board, review queue, split/merge tool	Week 6
Full dashboard figures: median resolution time, SLA compliance, reopen rate, backlog histogram (tiles currently show a labelled empty state)	Week 6
Live updates via Server-Sent Events	Week 6
Clustering evaluation: precision/recall/F1, merge-radius sweep, latency vs. scale, extent-cap ablation	Week 7
Rate limiting and anti-abuse	Month 2

Not built yet	Planned week
IEEE paper, presentation, recorded backup demo	Week 8

Known limitations right now

- **An acknowledged issue cannot move forward in the UI**, because the next step (assignment) is a supervisor tool due in week 6. The staff view says so explicitly. The full lifecycle already works through the API and is tested.
- **The report page's JavaScript is 297 KB gzipped**, over our revised budget. Form validation libraries (Zod and React Hook Form) account for about 96 KB (DD-031).
- **Free-tier cold start:** about 50 seconds including Render's container start after idle, although the application itself starts in 3.5 s.

---

20. Numbers at a glance

Metric	Value
Backend tests	**172** passing (real PostGIS)
Frontend tests	**58** passing
Backend source	~7,900 lines of Java
Frontend source	~8,400 lines of TypeScript/TSX
Database tables	11
Categories / departments / wards	10 / 7 / 4
Issue statuses	9
Design decisions recorded	57
Live corpus	110 issues from 250 reports
Report flow, open to ticket	3.0 s on a throttled slow connection
Application cold start	3.5 s
Concurrency guarantee	20 simultaneous reports → 1 issue

The live overdue count **rises by itself** as deadlines pass, so check it at `/api/v1/dashboard/summary` rather than quoting a fixed number.

---

21. Likely questions and answers

Q: What is the main technical contribution? The bounded incremental geo-clustering engine. It merges reports in real time using an accuracy-weighted centroid, an adaptive radius that tightens as evidence accumulates, a per-category ambiguous-band policy and an extent cap, and it stays correct under concurrent submissions.

Q: Why does the radius shrink as more reports come in? $\sigma = 1/\sqrt{\Sigma w}$ is the uncertainty of the cluster's position. More reports mean more certainty about where the problem really is, so a tighter radius avoids swallowing a different, nearby problem.

Q: Why do you need the extent cap? Without it, the centroid can move with every merge. Along a road, a chain of individually valid merges could turn one issue into a kilometre-long cluster. The cap limits the cluster's total spread relative to the category's radius.

Q: What happens if two people report the same thing at exactly the same time? An advisory lock on the (category, grid cell) makes them wait in line, and the second one re-reads the locked data before deciding. `ClusteringConcurrencyIT` proves that 20 simultaneous reports produce one issue.

Q: Why not just count reports for priority? One person could spam submissions. We count *distinct* reporters and use a logarithm, so volume from one street cannot dominate the city.

Q: How do you stop a department from closing complaints without fixing them? There is no staff transition to RESOLVED. It is missing from the transition table, the application refuses to start if someone adds it, a test checks all nine states, and citizens must verify.

Q: Why PostgreSQL + PostGIS instead of MongoDB or calculating distance in Java? Accurate distances on the Earth's surface, spatial indexes and polygon containment (which ward is this point in?) are built into PostGIS. Computing this in Java would be slower and error-prone, since lat/lng swaps and flat-earth maths are classic bugs.

Q: Why no H2 database in tests? H2 cannot run spatial SQL. Tests that don't use real PostGIS would not test the parts that matter.

Q: How do you know your tests actually work? We broke each protected behaviour on purpose and confirmed the test failed. We also run scripts against the real running server, because unit tests cannot see disagreements between frontend and backend.

Q: How will you evaluate the clustering? The seed generator produced ground-truth labels for every report. In week 7 we compare the engine's clusters to those labels to compute precision, recall and F1, sweep the merge radius, and switch off the extent cap to measure its effect.

Q: Why can anyone report without logging in? Requiring an account reduces reports, especially from people who distrust the system. The cost is that anonymous-only issues can't be citizen-verified, and we measure and publish that rather than hide it (DD-006, DD-017).

Sources in this repository: `docs/civictrack-project-report.md` (plan and specification), `docs/ARCHITECTURE.md` (diagrams), `docs/DESIGN-DECISIONS.md` (all 57 decisions), `docs/MONTH-1-STATUS.md`, `tools/README.md`.